PROJECT ABSTRACT

DISTRIBUTED MICROSERVICES BASED E-COMMERCE PLATFORM

OBJECTIVE

The objective of this project is to design and develop an online shopping platform using a distributed microservices architecture. Instead of placing all business functions inside one large application, the system separates user management, product catalog, shopping cart, orders, payments, and inventory into focused services. These services communicate through defined APIs and can be developed, tested, and maintained with less dependency on one another. The platform is intended to provide a clear shopping workflow for customers, product and order tools for sellers, and management controls for administrators.

EXISTING SYSTEM

The existing system uses a monolithic architecture to develop e-commerce applications. In this approach, all major functionalities, such as user management, product management, shopping cart, order processing, payment, and inventory management, are combined into a single application.

The following are the limitations of existing system:

Major Issues in the Existing System

1. High complexity due to a single application architecture.
2. Difficulty scaling individual functionalities.
3. Time-consuming deployment and updates.
4. Difficult maintenance as the application grows.
5. Risk of the entire application being affected by component failures.
6. Limited technology flexibility.
7. Database performance bottlenecks.
8. Difficulty integrating new features.

PROPOSED SYSTEM

The proposed system is an e-commerce application developed using microservices architecture. Instead of combining all functionalities into one large application, the system divides them into smaller, independent services. Each service is responsible for a specific function, such as user management, product management, product management, shopping cart, order processing, and payment management.

These services communicate with each other through APIs to provide a complete online shopping experience. This architecture improves scalability, flexibility, maintainability, and reliability.

Key features of proposed system

- Divides the application into independent services, making each functionality easier to manage.
- Scales only the required service when traffic increases, reducing unnecessary resource usage.
- Updates individual services without redeploying the entire application.
- Simplifies maintenance by keeping each service focused on a specific task.
- Reduces the impact of failures by isolating problems within individual services.
- Supports different technologies for different services based on their requirements.
- Maintains separate databases for services to reduce dependencies.
- Makes it easier to add new features without modifying the entire application.

IDENTIFIED USERS

The Distributed Microservices Based E-Commerce Platform consists of three main types of users: Customer, Vendor, and Administrator.

Roles and Responsibilities of Users

1. Customer

- Register and log in to the platform.
- View, search, and browse available products.
- View product details, prices, and availability.
- Add, remove, and update products in the shopping cart.
- Place orders and make payments.
- Track order status and delivery details.
- Cancel eligible orders and request returns according to platform policies.
- Provide product ratings and reviews.

2. Vendor / Seller

- Register as a vendor and obtain administrator approval if required.
- Add new products and manage existing product listings.
- Update product descriptions, prices, images, and categories.
- Maintain product stock and availability.
- View and manage customer orders assigned to them.
- Prepare orders and update fulfilment status.
- Monitor sales and order history.
- Handle product-related customer queries and return requests according to platform policies.

3. Administrator

- Manage customer and vendor accounts.
- Approve or reject vendor registration requests.
- Monitor and manage product listings.
- Oversee orders and resolve reported issues.
- Monitor payment status and authorized refund requests.
- View reports related to users, products, orders, and sales.
- Enforce platform rules and security policies.
- Monitor the overall functioning of the platform.

HARDWARE REQUIREMENTS

Component	Specification in your document
Processor	Intel Core i5, 8th generation, or AMD Ryzen 5 or higher
RAM	16 GB recommended
Storage	256 GB SSD minimum, with at least 30 GB free
Internet	Stable broadband connection

SOFTWARE REQUIREMENTS

- Operating System : Ubuntu 22.04 LTS or Windows 10/11 with Docker Desktop
- Programming Language : Python 3.11+
- Frontend: React.js 18 with Vite, Bootstrap, React Bootstrap
- Backend: Django REST Framework or FastAPI
- Database : MYSQL 8.0

TECHNOLOGY STACK

Programming Language:

- Python - Handles backend logic, user authentication, product management, and order processing.

Backend:

- Python- Used as the programming language to implement backend logic and business operations.
- Django- Used as the backend web framework to develop APIs, handle requests, manage database interactions, and implement authentication.
- Django with RestAPI- Used to develop RESTful APIs that enable communication between the frontend and backend services through HTTP requests and JSON data.

Frontend:

- HTML-Defines the structure of web pages.
- CSS- Designs and styles the website.
- JavaScript- Adds interactivity and connects the frontend with backend services.

Database :

- MYSQL- Stores customer, product, inventory, order, and payment details.

Authentication :

- JWT- Provides secure user authentication and role-based access.

CONCLUSION

The Distributed Microservices-Based E-Commerce Platform is designed to make online shopping easier, faster, and more efficient for customers and sellers. It allows customers to browse products, add items to their cart, place orders, and make payments, while sellers can manage their products and inventory. The administrator manages users and monitors the overall platform. By using microservices architecture, the application is divided into independent services, making it easier to develop, maintain, update, and scale as the number of users increases. This approach also helps reduce the impact of individual service failures and supports the addition of new features in the future. Overall, this project provides a flexible and scalable foundation for building modern e-commerce applications that can meet growing business and customer needs.